	PT RAVELWARE TECHNOLOGY – DIVISI SOFTWARE **STANDAR OPERASIONAL PROSEDUR** **BACKEND DEVELOPMENT**	Nomor	
		Tanggal Dibuat	Januari 2026
		Versi	1.0
		Tanngal Berlaku	
		Halaman	1 Dari 4

1. **Tujuan**
   Menjadi panduan dalam pengembangan backend aplikasi untuk memastikan konsistensi struktur kode, penamaan, manajemen database, penanganan error, logging, dan integrasi dengan sistem lain, sehingga menghasilkan layanan yang aman, terukur, dan mudah dipelihara.

2. **Ruang Lingkup**
   SOP ini berlaku untuk seluruh Backend Engineer di Divisi Software Development PT Ravelware Technology, mencakup aktivitas sejak inisialisasi proyek hingga deployment dan maintenance.

3. **Definisi & Istilah**
   a. Kebab-case: Penulisan nama dengan pemisah tanda hubung (-), contoh: user-service.
   b. CamelCase: Penulisan tanpa spasi, huruf pertama kata kedua dikapitalisasi, contoh: getUserById.
   c. Snake_case: Penulisan dengan underscore (_) sebagai pemisah, contoh: user_id.
   d. Migration: File skrip yang merekam perubahan skema database secara terkontrol.
   e. Artifact: Hasil build backend (misal: Docker image, executable) yang siap deploy.
   f. Winston: Library logging standar tim untuk pencatatan aktivitas sistem.

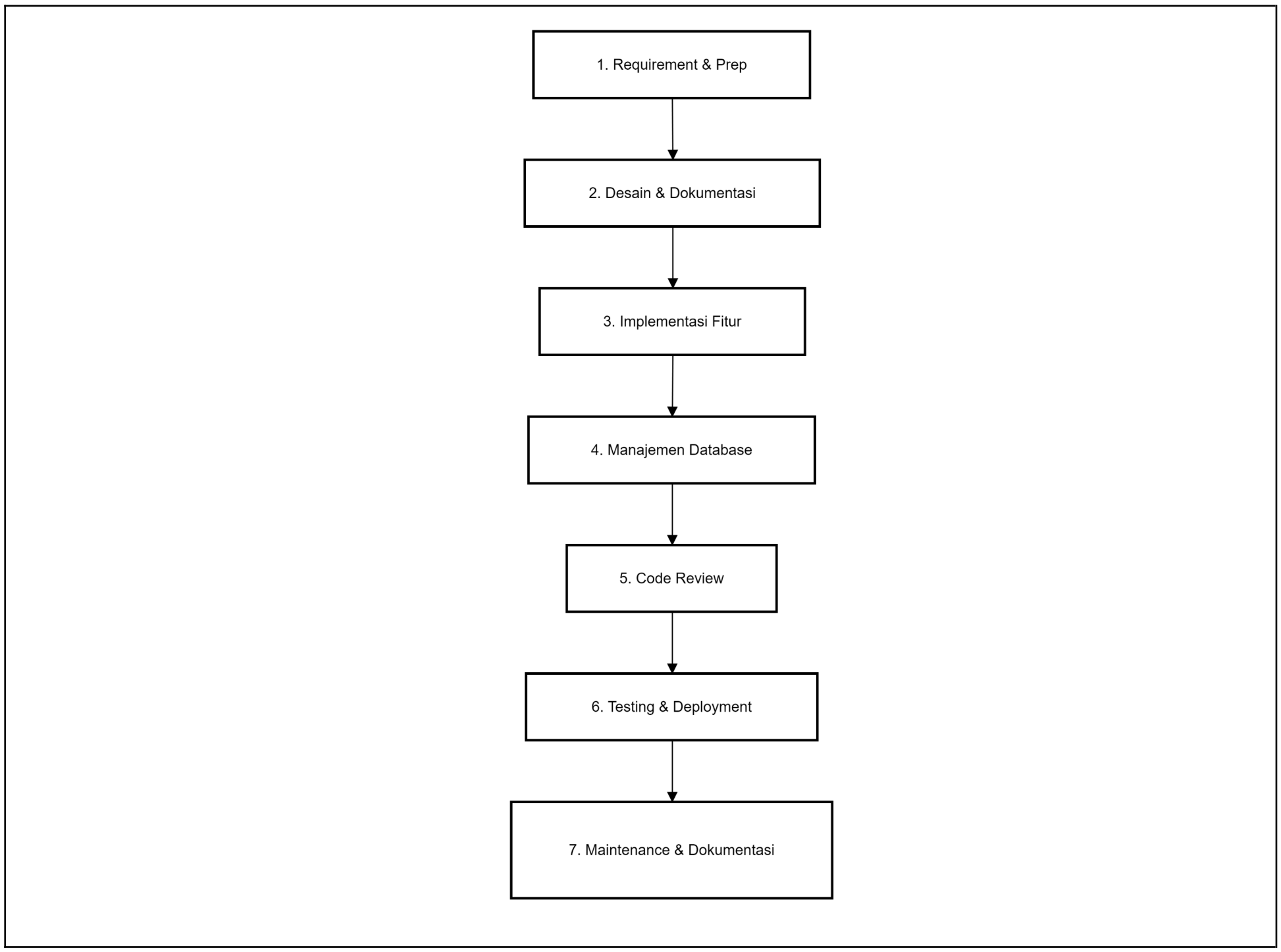
4. **Peran & Tanggung Jawab**
   a. Backend: Membangun layanan backend, menyediakan API, mengimplementasikan logika bisnis, dan mengelola database. Memastikan kode lolos analisis SonarQube sebelum merge serta bertanggung jawab atas keamanan, performa, dan perbaikan bug.
   b. Responsibility Person (RP): Memvalidasi kesesuaian implementasi backend dengan PRD dan API Specification. Menyetujui merge ke branch dev dan main, serta memastikan seluruh deliverables teknis memenuhi standar kualitas dan kebutuhan bisnis — tanpa terlibat dalam coding.

5. **Prosedur**

No	Aktivitas	Detail Aktivitas	Pelaksana	Deliverables
1	Requirement & Preparation	○ Inisiasi repository GitHub dengan template standar. ○ Gunakan struktur folder sesuai SOP. ○ Buat branch pribadi (nama developer). ○ Siapkan file .env.example, app.js, dan utils/ dasar.	Backend	○ Repository GitHub Backend ○ Struktur folder standar ○ Branch developer
2	Desain & Dokumentasi Teknis	○ Buat desain database (snake_case, tabel plural, kolom singular). ○ Susun API Specification (Postman/OpenAPI). ○ Koordinasi dengan FE dalam Technical Design Review (TDR).	Backend	○ Desain database awal ○ API Spec (request/response, error format) ○ Notulen TDR
3	Implementasi Fitur	○ Tulis service, controller, router dalam kebab-case. ○ Gunakan camelCase untuk fungsi & variabel. ○ Semua query DB melalui model/migration — tidak ada query langsung. ○ Implementasi error response sesuai format array of object. ○ Logging menggunakan Winston (level: error=0, info=1, dll).	Backend	○ Kode terstruktur sesuai SOP ○ Error handling standard ○ Log terklasifikasi
4	Manajemen Database	○ Setiap perubahan skema → buat file migration. ○ Nama database: singular snake_case. ○ Tidak ada folder query.	Backend	○ Migration files ○ Skema DB konsisten
5	Code Review & Integrasi	○ Push ke branch pribadi → buat Pull Request ke dev. ○ Setiap PR wajib melewati SonarQube (Quality Gate PASSED). ○ Pastikan integrasi dengan frontend berjalan lancar.	Backend, Frontend, & Responsibility Person	○ Laporan SonarQube ○ PR merged ke dev
6	Testing & Deployment	○ Perbaiki bug berdasarkan laporan Responsibility Person . ○ Setelah approval, merge dev → main. ○ Build artifact dari main. ○ Deploy production(on-premise/cloud).	Backend Engineer & Responsibility Person	○ Aplikasi running di production ○ Tag rilis GitHub (SemVer) ○ Status task “Done” di GitHub Project

7	Maintenance & Dokumentasi	○ Simpan log & storage di luar repo (terpusat, auto-purge). ○ Buat dokumentasi teknis (arsitektur, env setup, endpoint).	Backend & Responsibility Person	○ Dokumentasi teknis ○ Sistem logging & storage terkelola

6. Diagram Alir



7. Struktur dan Layout

```
├── src/
│   ├── controllers/        # Handler permintaan HTTP (tanpa prefix "Controller")
│   │   └── auth.js         # Contoh: authController → auth.js
│   │
│   ├── services/           # Logika bisnis & akses data
│   │   ├── auth/
│   │   │   ├── auth-service.js     # Implementasi layanan auth
│   │   │   └── helpers/            # Fungsi bantu khusus auth (misal: token generator)
│   │   │       └── token.js
│   │   └── user/
│   │       └── user-service.js
│   │
│   ├── routers/            # Definisi rute API (tanpa prefix "Router")
│   │   └── auth.js         # Contoh: authRouter → auth.js
│   │
│   ├── models/             # Model Sequelize (nama singular, sesuai tabel plural di DB)
│   │   └── User.js         # Mewakili tabel `users` di database
│   │
```

```
│   ├── migrations/          # File migrasi database (wajib untuk setiap perubahan skema)
│   │   └── 20260119000000-create-users-table.js
│   ├── utils/               # Utility global yang bisa dipakai di seluruh aplikasi
│   │   ├── response.js      # Standardisasi JSON response (success/error)
│   │   ├── string.js        # Fungsi seperti slugify, capitalize
│   │   └── database.js      # Helper pagination, query builder
│   ├── validations/         # Validasi input request (body, query, params)
│   │   └── auth-validation.js
│   ├── middlewares/         # Middleware Express (auth, logging, error handling)
│   │   └── auth-middleware.js
│   └── app.js               # Inisialisasi Express app, middleware, dan routing
│
├── .env.example             # Template variabel lingkungan (tidak boleh commit .env asli)
├── index.js                 # Entry point: jalankan server
├── package.json
└── README.md
```

a. controllers/: Folder yang berisi handler permintaan HTTP; bertugas menerima input dari klien, memanggil layanan yang sesuai, dan mengembalikan respons terstruktur — tanpa mengandung logika bisnis.
b. services/: Folder yang berisi logika bisnis aplikasi; setiap domain fungsional (misal: auth, user) memiliki subfolder sendiri yang mencakup implementasi layanan dan helper lokal.
c. routers/: Folder yang mendefinisikan rute API dan menghubungkannya ke controller yang sesuai; satu file per domain fungsional (misal: auth.js).
d. models/: Folder yang berisi representasi tabel database dalam bentuk model Sequelize; nama file menggunakan bentuk singular (misal: User.js) untuk merepresentasikan tabel plural di database (users).
e. migrations/: Folder yang menyimpan skrip perubahan skema database secara terkontrol; setiap perubahan struktur (CREATE, ALTER, DROP) wajib direkam sebagai file migrasi terpisah.
f. utils/: Folder yang berisi fungsi utilitas global yang dapat digunakan di seluruh aplikasi, seperti pembentuk respons standar, formatter string, atau helper pagination.
g. validations/: Folder yang berisi skema validasi input permintaan (body, query, params) untuk memastikan data masuk sesuai dengan ekspektasi sistem.
h. middlewares/: Folder yang berisi fungsi antara (middleware) untuk menangani otentikasi, logging, error handling, atau tugas lintas-cutting lainnya dalam alur request-response.
i. .env.example: File template yang berisi daftar variabel lingkungan yang diperlukan aplikasi; tidak boleh berisi nilai sensitif dan digunakan sebagai panduan untuk membuat file .env.
j. index.js: File titik masuk aplikasi yang menjalankan server Express pada port yang ditentukan dan mengimpor konfigurasi dari app.js.
k. app.js: File yang menginisialisasi instance Express, mendaftarkan middleware, mengatur routing, dan mengelola penanganan error global.

8. Catatan

Link Repository Backend: https://github.com/PT-Ravelware-Technology-Indonesia/new-sequelize-template-backend